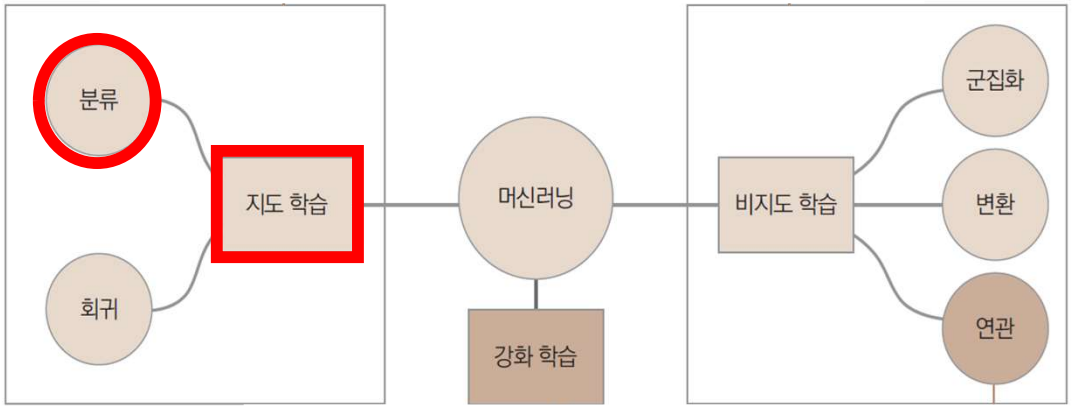
머신러닝 프로그래밍
Machine Learning Programming

Lecture 4-2 : 선형회귀

충북대학교
CHUNGBUK NATIONAL UNIVERSITY
충북대학교
산업인공지능연구센터

머신러닝 기법 : **지도학습**, 비지도학습, 강화학습



- 회귀 : 연속된 값을 측정하는 문제
  (ex) 중고차 가격 예측
- **분류 : 범주형 값을 측정하는 문제**
  (ex) 마케팅 문자를 보냈을 때 구매할 것 같은 고객을 예측하는 문제는 Yes/No 두 가지 범주로 표현

- 군집화(clustering) : 비슷한 데이터끼리 묶어주는 방법
  (ex) 소비 패턴이 유사한 카드 고객을 묶음.

- 변환(transformation) : 목적에 따라 데이터를 다른 형태로 바꿈.
  (ex) 변수 100개의 특성을 최대한 살려 변수 2개로 압축

- 연관(association) : 일명 장바구니 분석이라고도 하며, 유사한 구매 목록을 가진 고객끼리 비교하여, 서로의 장바구니에 담기지 않은 새로운 아이템을 추천을 하는 목적 등으로 사용.

분류(Classification) vs 회귀(Regression)

구분	분류 (Classification)	회귀 (Regression)
목표	데이터가 어떤 카테고리(범주)에 속하는지 예측 Which one ?	데이터의 연속적인 수치(값)를 예측 How much ?
결과값	이산적	연속적
예시	스팸 메일 차단, MNIST(0~9 숫자판별)	주가예측, 기온예측
평가방법	정밀도, 재현율, ROC	MSE(평균제곱오차), R^2 $Score$
문제확장	다중분류(3개이상)	다중회귀 (입력 특성이 여러 개)
심화분석	오류분석(혼동 행렬 시각화)	잔차분석(오차의 패턴확인)
복합예측	다중 레이블 / 다중출력	다변량 회귀 (출력값이 여러 개인 경우)
핵심 모델	로지스틱 회귀(4주차), 의사결정트리(5주차) K-최근접이웃(6주차), 서포트벡터벡신(7주차)	선형회귀(9주차), 앙상블(10주차)

✓ 로지스틱 회귀 : 작동원리가 선형회귀와 거의 유사, but 시그모이드(sigmoid) 함수에 통과시켜 0과 1사이의 확률값으로 변환하하여 분류하기 때문에 분류로 봄

선형회귀의 개념

- 선형회귀(Linear Regression) : 종속변수 y와 한 개 이상의 독립변수 x와의 선형 상관관계를 모델링하는 회귀분석 기법
- 기존 데이터를 활용해 연속형 변수값을 예측
- $y = ax + b$꼴의 수식을 만들고 a와 b의 값을 찾아냄

선형 회귀

- 앞으로 개봉할 영화 예상 관객 수 y를 예측하는 문제

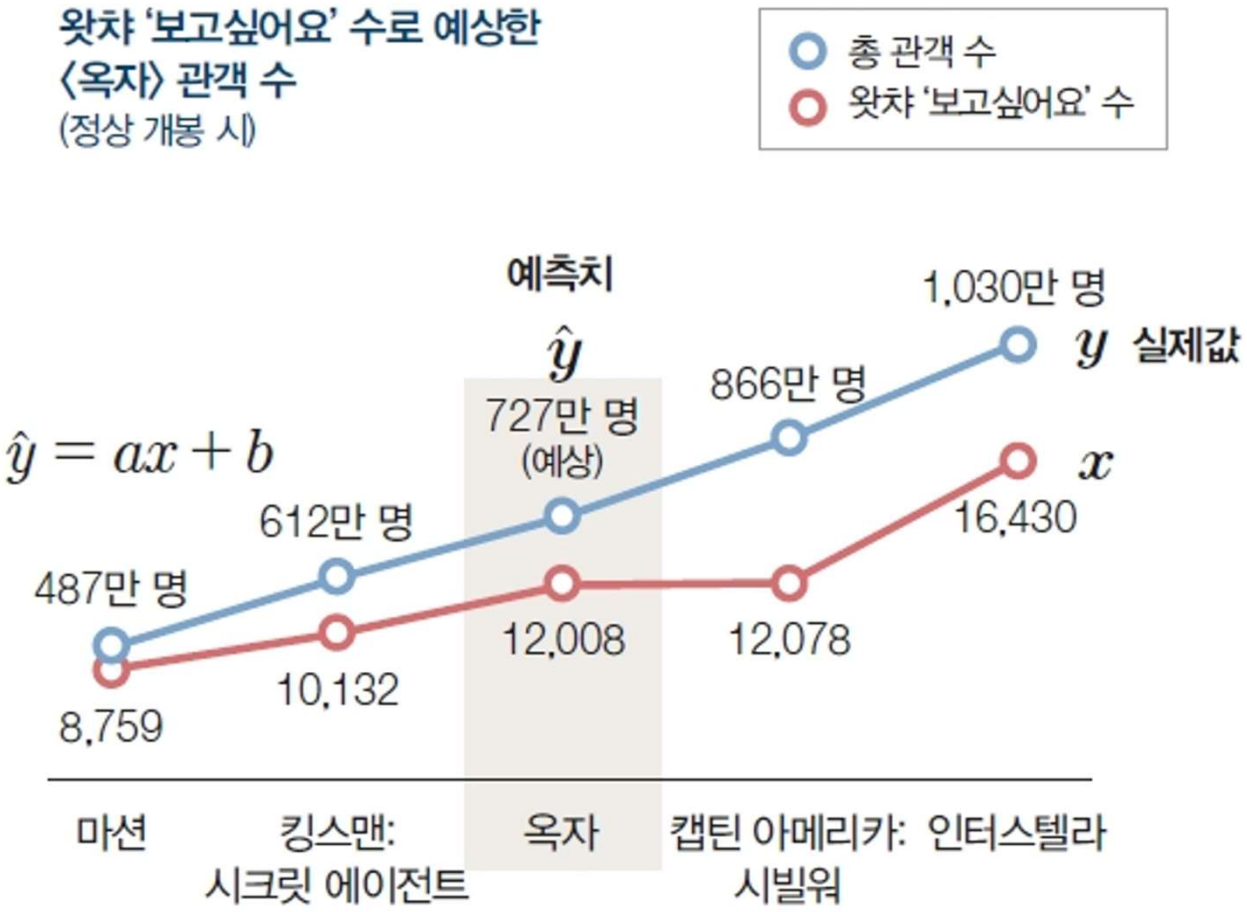


그림1. 왓챠 '보고싶어요 ' 수로 예상한 옥자 관객수

선형 회귀

- 실제 관객 수를 y로 표현하여 좌표평면 상에 나타냄

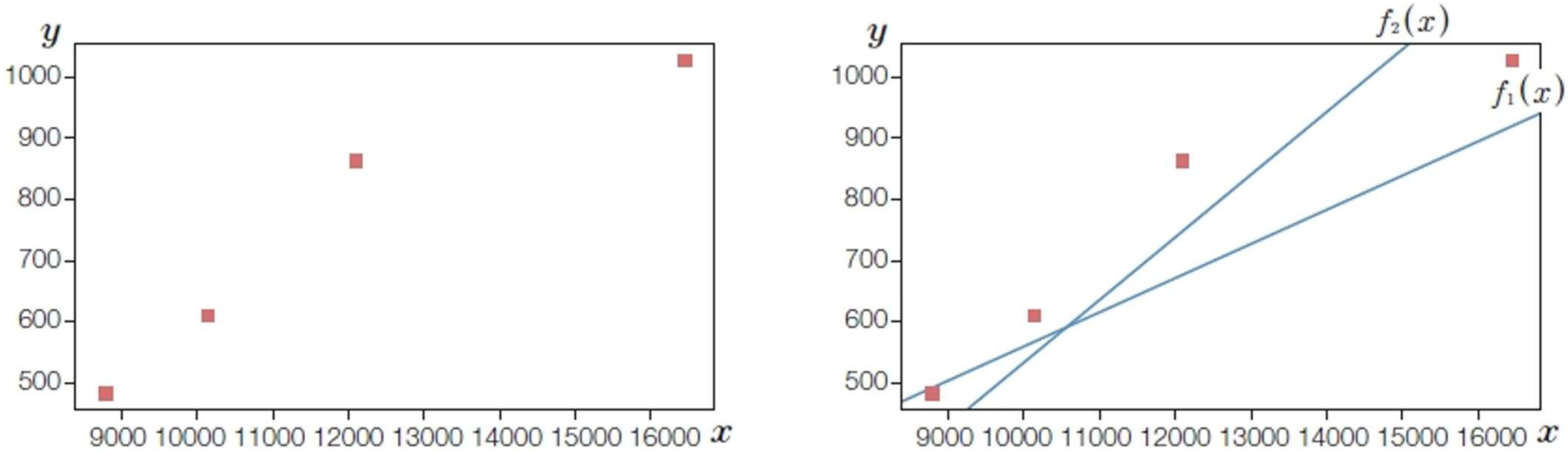
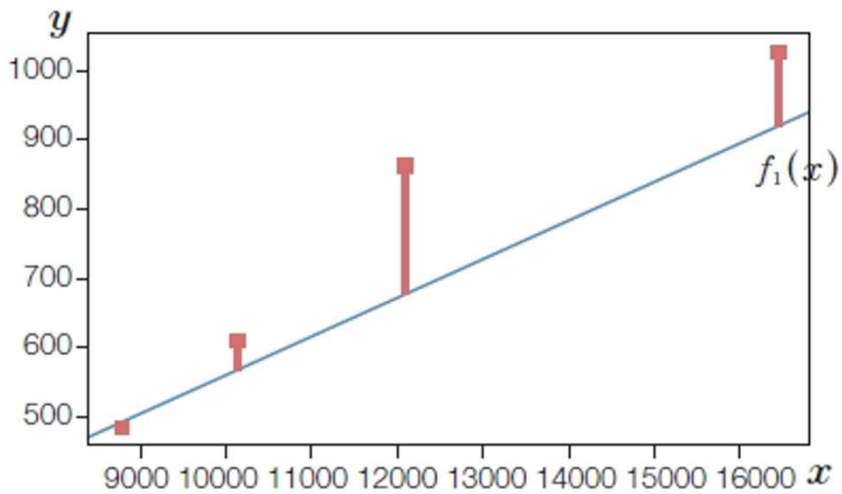
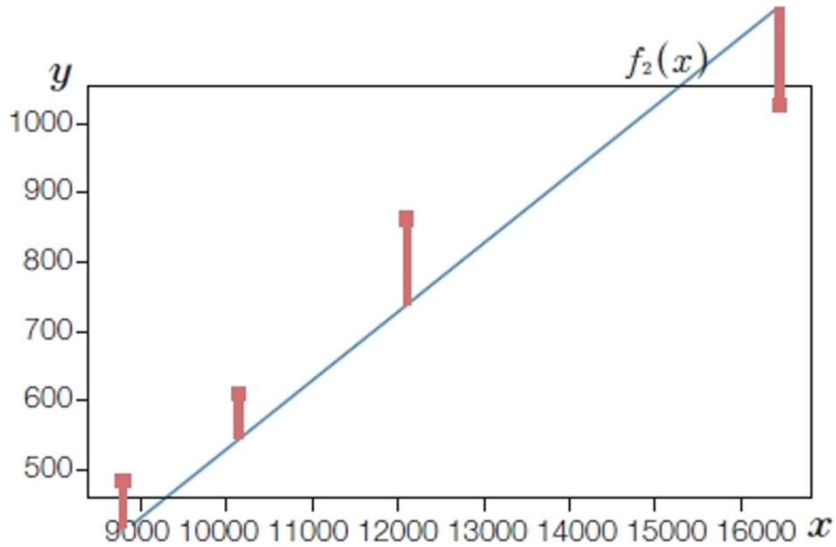


그림2. 좌표평면에 작성한 2개의 직선

선형 회귀

- 두 그래프 중 어떤 것이 기존 데이터를 '잘 표현하는가'
- 예측값이 실제값 대비 차이가 많이 나지 않는 그래프





예측 함수와 실제값 간의 차이

- 예측 함수는 예측값과 실제값 간의 차이를 최소화하는 방향으로
- 데이터 n개 중 i번째 데이터의 y값에 대한 실제값과 예측값의 차이 $\hat{y}^i - y^i$
- 데이터가 5개 있을 때 5개 데이터의 오차의 합

$$(\hat{y}^{(1)} - y^{(1)}) + (\hat{y}^{(2)} - y^{(2)}) + (\hat{y}^{(3)} - y^{(3)}) + (\hat{y}^{(4)} - y^{(4)}) + (\hat{y}^{(5)} - y^{(5)})$$

  - 오차 값들이 음수와 양수로 나왔을 때 값들 간의 차이가 상쇄되어 0으로 계산될 수 있음

선형 회귀

- 값의 제곱을 사용하여 오차의 합을 표현

$$(\hat{y}^{(1)}-y^{(1)})^2+(\hat{y}^{(2)}-y^{(2)})^2+(\hat{y}^{(3)}-y^{(3)})^2+(\hat{y}^{(4)}-y^{(4)})^2+(\hat{y}^{(5)}-y^{(5)})^2$$

$$\sum_{i=1}^{n}(\hat{y}^{(i)}-y^{(i)})^2$$

- 같은 식을 행렬로 표현

$$\hat{y}=\begin{bmatrix} w_1\times 8759+w_0 \\ w_1\times 10132+w_0 \\ w_1\times 12078+w_0 \\ w_1\times 16430+w_0 \end{bmatrix} \qquad y=\begin{bmatrix} 487 \\ 612 \\ 866 \\ 1030 \end{bmatrix}$$

$$(\hat{y}-y)^2=\begin{bmatrix} (w_1\times 8759+w_0-487)^2 \\ (w_1\times 10132+w_0-612)^2 \\ (w_1\times 12078+w_0-866)^2 \\ (w_1\times 16430+w_0-1030)^2 \end{bmatrix}$$

선형 회귀

- 제곱 오차(square error) : $(\hat{y} - y)^2$ 로 예측값과 실제값의 제곱을 표시하여 오차를 나타냄

- 제곱 오차를 최소화하는 w_0와 w_1을 찾아야 함

$$\sum_{i=1}^{n} \left(w_i x^{(i)} + w_0 \times 1 - y^{(i)} \right)^2$$

선형 회귀

- 제곱 오차(square error) : $(\hat{y} - y)^2$ 로 예측값과 실제값의 제곱을 표시하여 오차를 나타냄

- 제곱 오차를 최소화하는 w_0와 w_1을 찾아야 함

$$\sum_{i=1}^{n} \left(w_i x^{(i)} + w_0 \times 1 - y^{(i)} \right)^2$$

선형 회귀의 기초 수식

머신러닝을 지탱하는 두 기둥

1. 가설함수 (Hypothesis)

데이터를 넣었을 때 예측값을 만들어내는
"예측 레시피"

$$h_\theta(x) = f(x, \theta)$$

2. 비용함수 (Cost Function)

예측값이 실제와 얼마나 다른지 평가하는
"맛 평가 점수"

$$J(\theta) = \mathrm{Error}(y, \hat{y})$$

선형 회귀의 기초 수식

가설함수: 알고리즘별 '예측 레시피'

알고리즘	가설함수의 형태 (h)	예측 방식
선형 회귀	$Wx + b$	연속적인 수치 (직선)
로지스틱 회귀	$\sigma(Wx + b)$	0과 1 사이의 확률 (S곡선)
소프트맥스 회귀	$\text{Softmax}(z)$	여러 클래스의 확률 분포

💡 각 기법들은 자신의 목적에 맞는 특수한 형태의 가설함수를 활용합니다.

선형 회귀의 기초 수식

비용함수: 알고리즘별 '맛 평가'

회귀 (Regression)

예측치와 실제치의 **거리**를 잽니다.

👉 MSE (평균 제곱 오차)

분류 (Classification)

예측 확률과 실제 확률의 **차이**를 잽니다.

👉 Cross Entropy (교차 엔트로피)

비용함수의 결과값이 **0**에 가까울수록 "공부를 아주 잘한 모델"입니다.

선형 회귀의 기초 수식

머신러닝의 전체 시나리오

Step 1
가설 설정
(Recipe)

→

Step 2
비용 계산
(Scoring)

→

Step 3
최적화/학습
(Revision)

"비용이 최소가 될 때까지 **가중치(θ)**를 조금씩 수정하며 반복합니다."

선형 회귀의 기초 수식

직관적인 비유로 정리하기

개념	비유	설명
가설함수 (h)	**요리 레시피**	재료(x)를 넣어 요리(y)를 만드는 법
비용함수 (L)	**시식평 점수**	예측이 실제와 얼마나 다른지 수치화
학습 (Learning)	**레시피 수정**	점수를 높이려고 조미료(θ)를 조절하는 과정

선형 회귀의 기초 수식

비용함수의 개념

- 비용함수(cost function) : 머신러닝에서 최소화해야 할 예측값과 실제값의 차이
- 가설함수(hypothesis function) : 예측값을 예측하는 함수

$$f(x) = h_\theta(x)$$

- 함수 입력값은 x이고
  함수에서 결정할 것은 θ로, 가중치(weight) 값인 w_n

선형 회귀의 기초 수식

- 비용함수가 두 개의 가중치 값으로 결정됨

$$J(w_0, w_1) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - (y^{(i)}))^2$$

  - 잔차의 제곱합(Error sum of squares) : 예측값인 가설함수와 실제값인 y값 간의 차이를 제곱해서 모두 합함
    - 총 데이터는 m개가 존재하고 각 데이터의 예측값과 실제값을 뺀 후 제곱한 값들을 모두 합한 값
  - 손실함수(loss function) : 비용함수에서 잔차의 제곱합 부분
  - 평균 제곱 오차(mean squared error, MSE) : 잔차의 제곱합을 2m으로 나눈 값

선형 회귀의 기초 수식

4-4.linear.py

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

np.random.seed(42)
X = 100 * np.random.rand(100, 1)  # 100 random values between 0 and 100
y = 5 * X + 10 + np.random.randn(100, 1) * 10

# 2. Create and Train the Linear Regression Model
model = LinearRegression()
model.fit(X, y)

# 3. Check Learned Parameters (Weights and Bias)
w1 = model.coef_[0][0]    # Slope (Weight)
w0 = model.intercept_[0]  # Intercept (Bias)

print(f"=== Training Results ===")
print(f"Estimated Slope (w1): {w1:.2f}")
print(f"Estimated Intercept (w0): {w0:.2f}")
print(f"Final Equation: y = {w1:.2f}x + {w0:.2f}")

# 4. Prediction and Evaluation
y_pred = model.predict(X)
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)

print(f"\n=== Model Evaluation ===")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R-squared (R2 Score): {r2:.2f}")

# 5. Visualization with English Labels
plt.figure(figsize=(10, 6))

# Plot actual data points
plt.scatter(X, y, color='blue', alpha=0.6, label='Actual Data')

# Plot the learned regression line
plt.plot( *args: X, y_pred, color='red', linewidth=2, label='Regression Line')

# Adding graph details in English
plt.title( label: "Linear Regression: Watcha Likes vs Audience", fontsize=14)
plt.xlabel( xlabel: "Watcha 'Like' Count", fontsize=12)
plt.ylabel( ylabel: "Total Audience Count", fontsize=12)
plt.legend()
plt.grid( visible: True, linestyle='--', alpha=0.6)

# Display the plot
plt.show()
```

선형 회귀의 기초 수식

4-5.linear_real.py

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# 1. 데이터 불러오기 (경로 단순화)
file_path = r"D:\My_project\4 weeks\Salary_Data.csv"
data = pd.read_csv(file_path)

# 2. 데이터 분리 (X: 경력, y: 연봉)
X = data[['YearsExperience']]
y = data['Salary']

# 3. 모델 생성 및 학습
model = LinearRegression()
model.fit(X, y)

# 4. 학습 결과 확인
w1 = model.coef_[0]      # 기울기 (1년당 인상액)
w0 = model.intercept_   # 절편 (초봉)

print(f"=== AI 분석 결과 ===")
print(f"공식: Salary = {w1:.2f} * Experience + {w0:.2f}")
print(f"1) 연봉 인상액: ${w1:.2f}")
print(f"2) 예상 초봉: ${w0:.2f}")

# 5. 모델 평가 및 미래 예측
y_pred = model.predict(X)
print(f"3) 정확도(R2): {r2_score(y, y_pred):.4f}")

# 경력 15년차 예측 (Feature names Warning 방지를 위해 DataFrame 사용)
X_new = pd.DataFrame( data: [[15]], columns=['YearsExperience'])
pred_15 = model.predict(X_new)
print(f"4) 15년차 예상 연봉: ${pred_15[0]:,.2f}")

# 6. 시각화
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot( *args: X, y_pred, color='red', linewidth=2, label='Regression Line')

plt.title( label: "Salary vs Experience Analysis", fontsize=15)
plt.xlabel("Years of Experience")
plt.ylabel("Salary ($)")
plt.legend()
plt.grid( visible: True, alpha=0.3)
plt.show()
```